audio.dev
CONFERENCE
BRISTOL & ONLINE 2026

Workshop:

The Audio Interface ‘Escape Room’

Debugging an Audio Product, One Clue at a Time

Ross Chisholm & Alex Middleton-Dalby

Before we start

Get the latest workshop code

Already cloned it? Pull the full workshop and rebuild, from the repository folder:

```
git pull
cmake -DEX=1 -DMODE=start -P scripts/exercise.cmake
```

Not set up yet? Follow the README, then run the same build command:

github.com/FocusriteGroup/the-audio-interface-escape-room-adc-26

The first build downloads and compiles JUCE: start it now, during the introduction.

Workshop objectives

Track down issues in a number of places within a small two-input, two-output audio app.

01

Reproduce

Follow the steps and confirm the problem.

02

Measure/Investigate

Identify the point in the app where the issue starts.

03

Verify

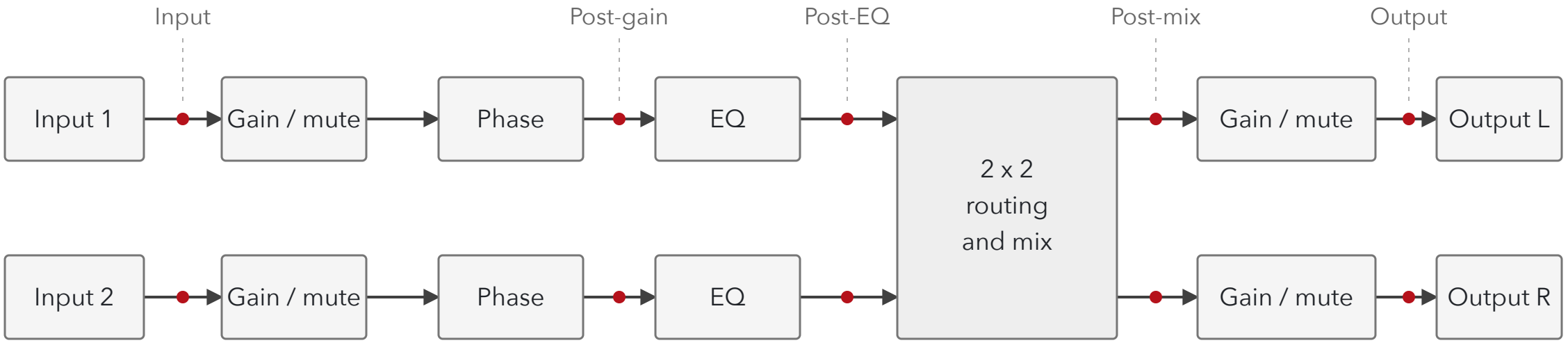
Find the cause and confirm it is fixed.

Agenda

Section	
Getting started	Introduction, setup and project outline
Signal investigations	Exercises 1-3: test tone, right channel and no sound in mono
Break	15 minutes
System investigations	Exercises 4-5: playback buzz and saved setup
Final Puzzle	Combined exercise (recording)

Application overview

Signal path



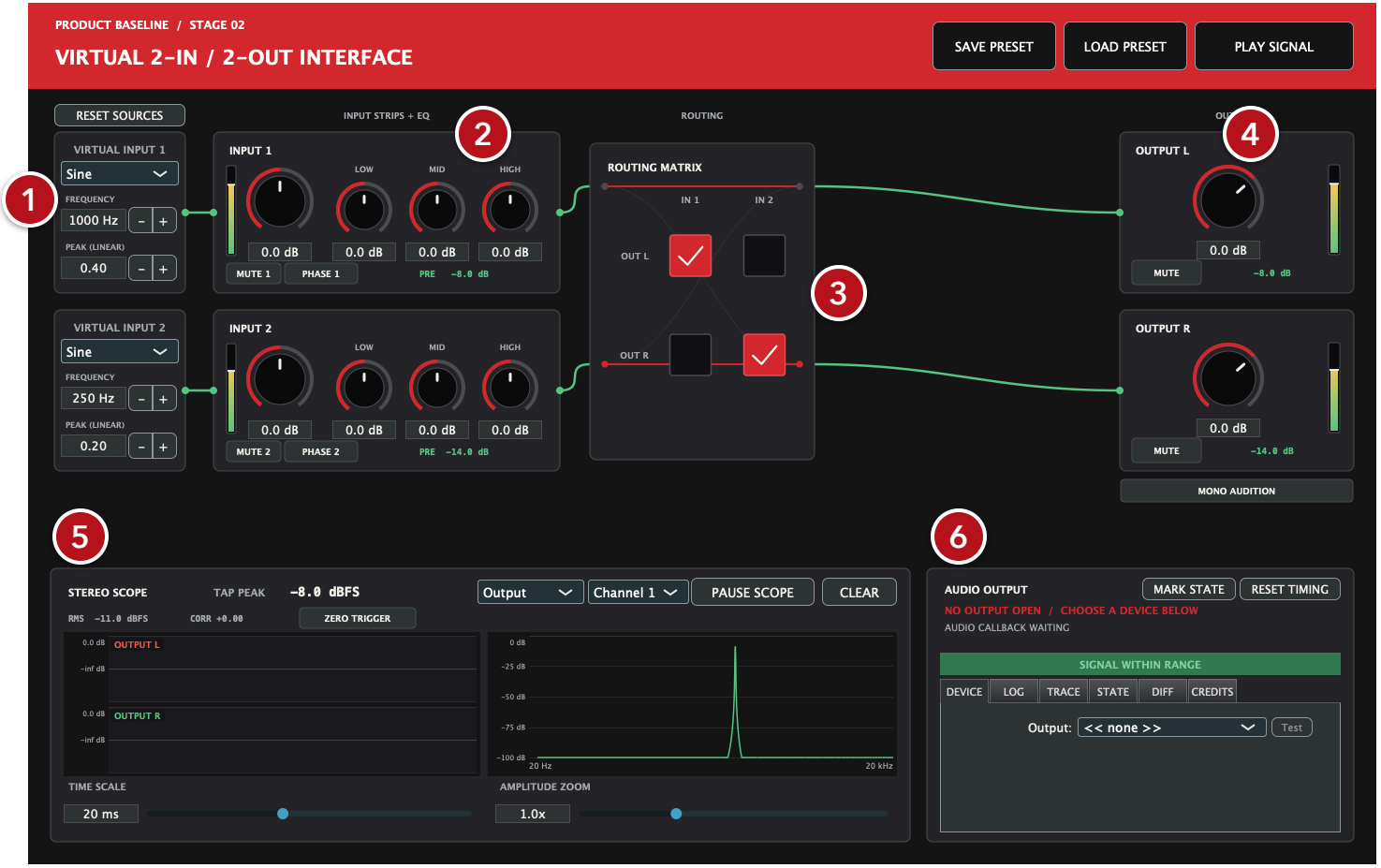
- Analysis taps on both channels; post-gain is measured after the phase invert.

Live playback and the tests use the same C++ processing core.

Virtual inputs. System stereo output. No external interface required.

Application overview

Application controls



1. **Sources**
   Waveform, frequency and peak.

2. **Input strips**
   Gain, mute, phase and EQ.

3. **Routing**
   Send either input to either output.

4. **Outputs**
   Gain, mute, meters and mono.

5. **Analysis**
   Tap, scope and spectrum.

6. **Diagnostics**
   Device, timing, log, trace, state and diff.

Application overview

Diagnostic tools

Tool	What can it be used for?
Analysis taps and scope	Compare waveforms between processing stages in time domain
Peak, RMS, clip count	Measure signal level and above-full-scale samples.
Correlation and spectrum	Compare stereo correlation and frequency domain content.
Callback timing	Compare duration, budget and missed deadlines.
Logger (LOG)	Read ordered event messages and recorded values.
Debugger (VS Code)	Step through the focused check or the running app.

Live demo

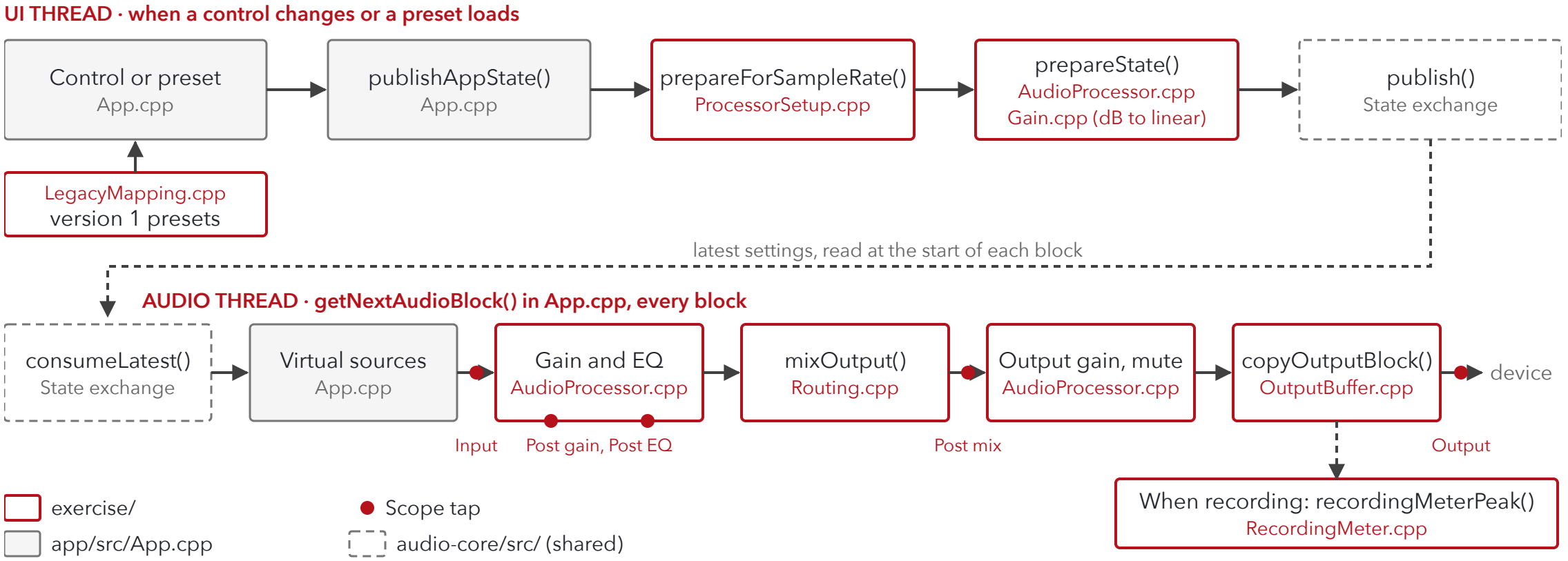
Project overview

Where to work

Location	Participant use
`README.md`	Project setup and build commands.
`missions/<exercise>/README.md`	Exercise instructions, commands and criteria.
`exercise/`	All the processing code for the current exercise.
`tests/`	Contract, scenario and regression tests.
`audio-core/src/`	Shared headers, analysis and test fixtures.
`app/src/`	JUCE interface, audio callback and diagnostics.
`assets/`	Audio and preset fixtures.

Build and run: `scripts/exercise.cmake` picks the exercise and mode.
After each edit: `cmake -P scripts/check.cmake` reruns the exercise's test.\

How App.cpp calls the exercise code



Setup

Exercise 1 - Getting Started

```
cmake -DEX=1 -DMODE=start -P scripts/exercise.cmake
```

This builds the app, opens it and ends with the exercise's test result.

The test should fail: **+6.0206 dB should double amplitude**.

After each edit, rerun just the test:

```
cmake -P scripts/check.cmake
```

Find the app you just built

Build directory: `build/workshop/app/`

Output	Path inside this build directory
macOS app	`bin/Fix It Before Launch.app`
Windows app	`bin/Fix It Before Launch.exe`
Linux app	`bin/Fix It Before Launch`
Test executables	`tests/` (run with `cmake -P scripts/check.cmake`)

If the app doesn't open by itself (Linux), launch it from `bin/`.

To hear an edit in the app: **close the app and run the same command again**.

Investigation method

Locate the first incorrect stage

1. **Reproduce** the issue.
2. **Compare** the expected and observed behaviour.
3. **Identify** which stage the issue occurs in.
4. **Repair** the root cause of the issue
5. **Verify** the fix and explain the result.

Exercise 1 / Bug report

The test tone sounds distorted

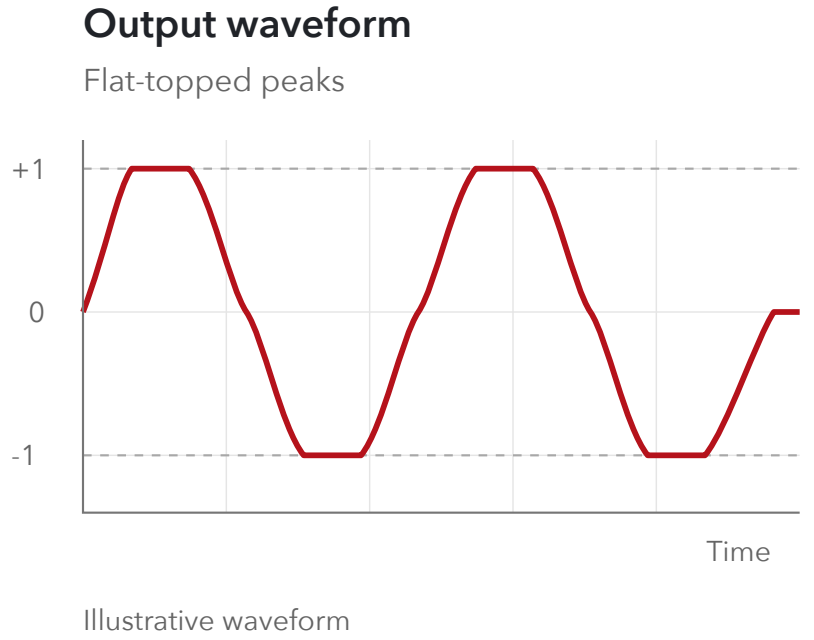
Reported setup: supplied settings, headphones.

1. Open the app and start the test tone. It sounds fine.
2. Turn Input 1 gain up to +6 dB.
3. Listen briefly, then stop playback.

Expected: a louder but still clean tone. +6 dB shouldn't clip.

Actual: a distorted sound on every attempt.

Frequency: repeatable.



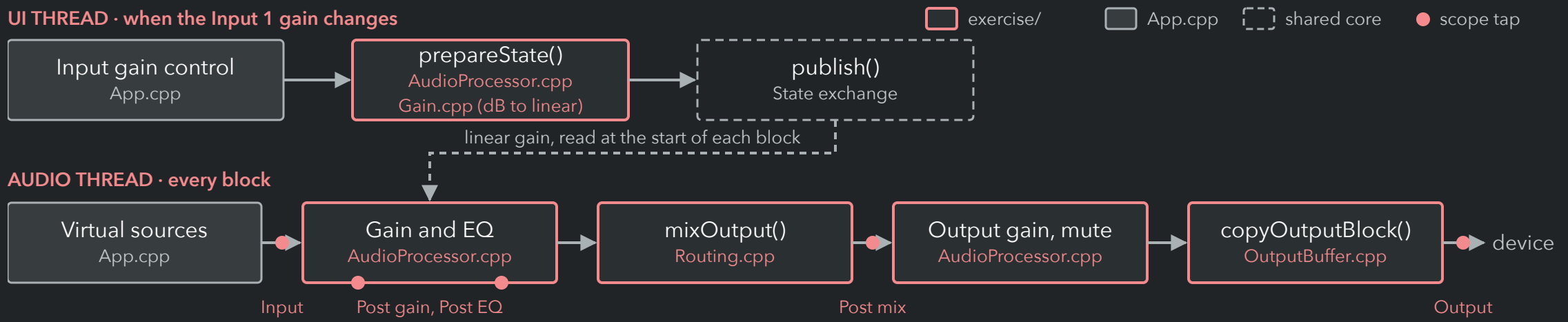
Decibels, amplitude and dBFS

- **Amplitude ratio** (sample values): $10^{\mathrm{dB}/20}$
- **Power ratio** (energy): $10^{\mathrm{dB}/10}$
- Power is proportional to amplitude²
- **dBFS** $= 20\log_{10}(\text{peak sample value})$. A peak of 1.0 is 0 dBFS; anything above it clips at the output. Internally the app processes in floating point but DACs cannot reproduce anything higher than 0 dBFS.

Change	Amplitude	Power
0 dB	×1	×1
+6.02 dB	×2	×4
−6.02 dB	×0.5	×0.25
+20 dB	×10	×100

Exercise 1 / Investigate

Find the first stage where the tone goes wrong



REPRODUCE

1. Source **Sine**, 1 kHz, peak 0.4 (RESET SOURCES).
2. Play, then raise Input 1 from **0 dB** to **+6.0 dB**.
3. Compare the **Input** and **Post gain** taps.
4. Note meter values and the clip count.

THEORY

- Amplitude $10^{\mathrm{dB}/20}$, power $10^{\mathrm{dB}/10}$.
- +6.02 dB should double the amplitude.
- A peak above 1.0 (0 dBFS) clips at the output.

After each edit: `cmake -P scripts/check.cmake` or rebuild the app.

Exercise 1 / Hint 1

Follow The Signal

With the gain for input 1 set to +6.0dB, the signal is clipping.

Change the 'tap' point of the scope. Can you work out at what stage the clipping is introduced?

Exercise 1 / Hint 2

+6dB should double the amplitude

A gain of `+6dB` should multiply the amplitude by two.

With the pre-gain peak set to `0.4`, we'd expect the post-gain peak to be roughly `0.8`: instead, it's clipping. Is something wrong with our gain calculation?

Exercise 1 / Hint 3

Gain.cpp should use the amplitude gain formula

Power ratios and amplitude ratios use different formulas. Audio samples represent amplitude, so that's the formula we should be using.

Compare this against the code in `Gain.cpp`.

Amplitude gain formula:

$$10^{\mathrm{dB}/20}$$

Exercise 1 / Answer

Power conversion applied to amplitude

Measurement at +6.0206 dB	Expected	Broken
Linear gain	2.0	4.0
Output peak (input 0.4)	0.8	1.6

```
std::pow(10.0f, decibels / 10.0f)   // a power ratio, applied to samples
```

At +6.0206 dB this returns **four**, not two. The source is unchanged; the error is in the dB-to-linear conversion.

Exercise 1 / Fix

Use the amplitude-ratio formula

```
std::pow(10.0f, decibels / 20.0f)
```

- 0 dB = ×1, +6.02 dB = ×2, −6.02 dB = ×0.5.
- Centralise conversion functions like this to reuse across app.
- Use useful variable and function names to make it clear which domain it's working in.

Optional catch-up: stop and close the app; build, test and open the solution.

```
cmake -DEX=1 -DMODE=solution -P scripts/exercise.cmake
```

Next: close this app, then open Exercise 2.

```
cmake -DEX=2 -DMODE=start -P scripts/exercise.cmake
```

Exercise 2 / Bug report

No sound in the right channel

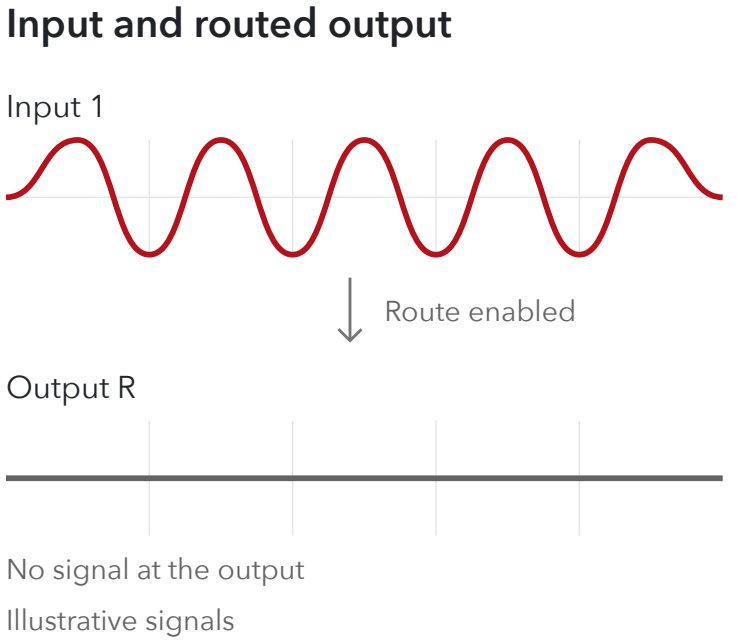
Reported setup: headphones; audio on both inputs.

1. Start playback on both inputs.
2. Route Input 1 to Output R only.
3. Listen to each headphone channel.

Expected: Input 1 on the right; nothing on the left.

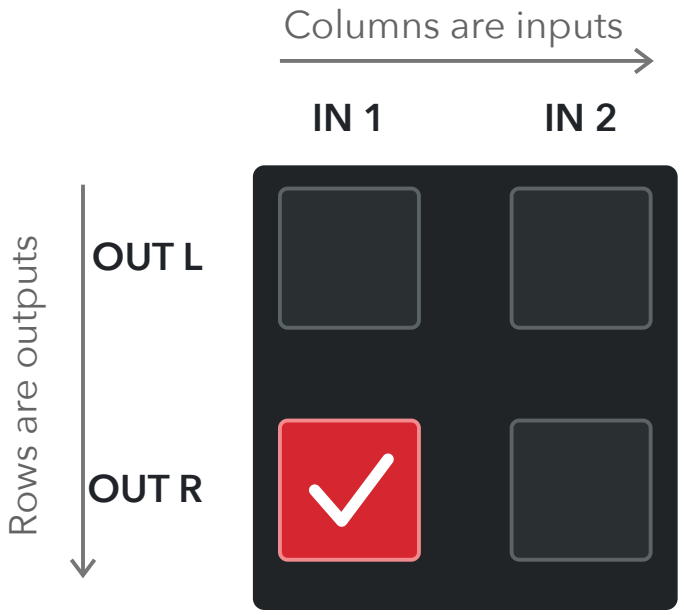
Actual: the right is silent and the left plays Input 2, which isn't routed.

Frequency: every attempt with these settings.



Routing matrices

- The **routing matrix** holds a gain for each output and input.
- Each output sums every input times its gain: $y_o[n] = \sum_i R_{o,i}\, x_i[n]$
- Code counts from 0: Input 1 is index 0, Output R is index 1.
- **Give each input its own tone** to hear and see where each one goes.
- The grid shows the intended route; the taps show what is happening.

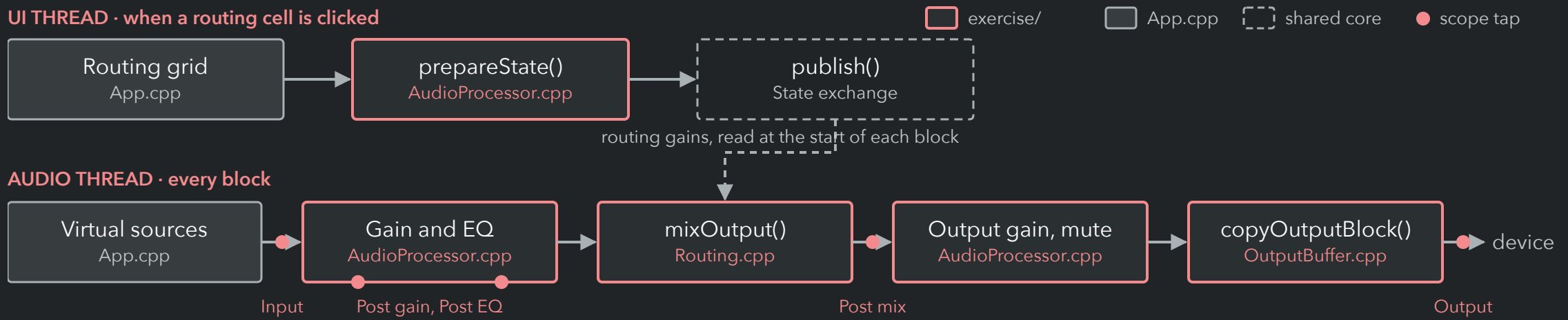


Start route: **Input 1 → Output R**

Both inputs play: IN 1 at 1 kHz, IN 2 at 250 Hz

Exercise 2 / Investigate

Find where the audio disappears



REPRODUCE

1. Start state: Input 1 at 1 kHz, Input 2 at 250 Hz.
2. Keep the start route: Input 1 → Output R only.
3. Audio should be present on Output R.

THEORY

- Grid rows are outputs; columns are inputs.
- $y_o[n] = \sum_i R_{o,i}\, x_i[n]$, indexes from 0.
- 1 kHz is Input 1; 250 Hz is Input 2.

After each edit: `cmake -P scripts/check.cmake` or rebuild the app.

Exercise 2 / Hint 1

Follow the lines on the routing matrix

Take note of the settings for the two inputs (you could even make them more distinct from each other if you like).

Toggle the checkboxes in the routing matrix and follow the red lines. Are you hearing what you'd expect in each output channel given what's showing on the UI?

Exercise 2 / Hint 2

Look for where the inputs are mixed together

Something's going wrong at the routing stage. Look for the code where the routing state is accessed, and the input signals are mixed together.

Think about what could be causing the behavior we're seeing.

Focusrite

Exercise 2 / Hint 3

Look at how the routing state is stored

Look for where the routing state (`state.routing` in `Routing.cpp`) is defined. What's the structure of the data? Does that match with how it's being accessed?

You could even search for `state.routing` to see if there are other usages that might provide some clues.

Focusrite

Exercise 2 / Answer

The matrix is transposed

Post mix tap, start route	Expected	Broken
Output L	**Silent**	**250 Hz (Input 2)**
Output R	**1 kHz (Input 1)**	**Silent**

```
mixed += processedInputs[inputIndex] * state.routing[inputIndex][outputIndex];
```

State stores **[output][input]**; the audio processing reads **[input][output]**, so Input 1 → Output R plays as Input 2 → Output L.

Focusrite

Exercise 2 / Fix

Use output-first indexing

```
mixed += processedInputs[inputIndex] * state.routing[outputIndex][inputIndex];
```

- Test all four cells, one signal at a time.
- Identity routing passed: as the matrix is transposed it gives the same result.
- Assert the wanted and the silent output.
- Measure the audio, not what the UI shows.

Optional catch-up: stop and close the app; build, test and open the solution.

```
cmake -DEX=2 -DMODE=solution -P scripts/exercise.cmake
```

Next: close this app, then open Exercise 3.

```
cmake -DEX=3 -DMODE=start -P scripts/exercise.cmake
```

Exercise 3 / Bug report

No sound when in mono

Reported setup: two matching input signals, headphones.

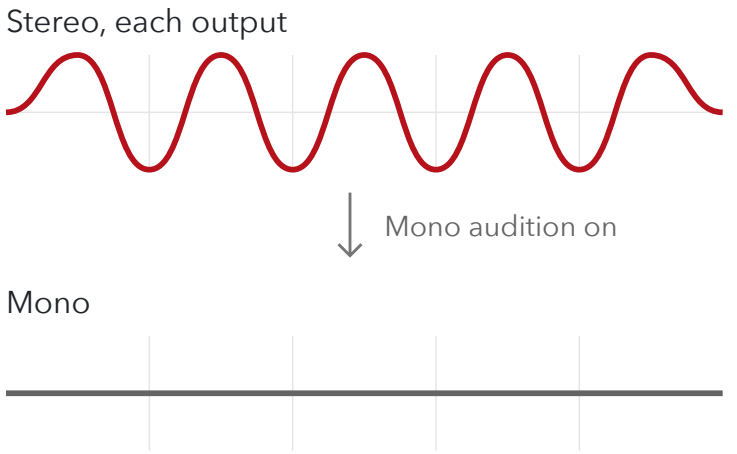
1. Play the two signals.
2. Switch mono audition on.
3. Switch back to stereo and compare.

Expected: the same content remains audible.

Actual: silence in mono, although both outputs play in stereo.

Frequency: repeatable.

Stereo and mono



Illustrative waveforms

Exercise 3 / Theory

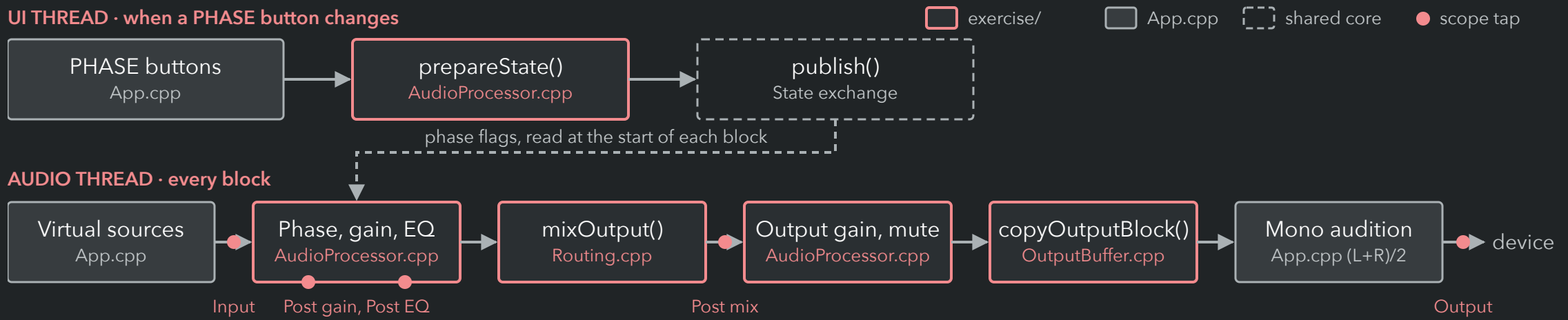
Phase, correlation and mono

- **Phase invert** multiplies every sample by −1. One channel alone sounds the same.
- **Mono** averages the outputs: $m = (L + R)/2$. If $R = -L$, then $m = 0$.
- **Correlation:** +1 identical, 0 unrelated, −1 opposite. Only meaningful if you know the source.
- Per-channel peaks and spectra can't show a problem *between* channels. Compare both on one time axis.

Same sine on L and R	Aligned	One inverted
Left peak	0.4	0.4
Right peak	0.4	0.4
Correlation	+1	−1
Mono peak	0.4	0

Exercise 3 / Investigate

Explain why the sound disappears in mono



REPRODUCE

1. Start state: the same 120 Hz sine on both inputs.
2. Scope both outputs together; note the correlation.
3. Turn on **MONO AUDITION**, below the outputs.

THEORY

- Phase invert: every sample × −1.
- Mono = $(L + R)/2$
- Correlation: +1 aligned, −1 opposite.

After each edit: `cmake -P scripts/check.cmake` or rebuild the app.

Exercise 3 / Hint 1

Look at the two channels in the scope

Play the signal, pause the scope, and then look at the two channels shown in red and green. What do you notice about them?

Try moving the tap point on the scope. What changes?

Focusrite

Exercise 3 / Hint 2

Investigate the phase buttons on the input strips

Try pressing the PHASE 1 and PHASE 2 buttons. Are they doing what we'd expect?

Look for where the phase state is processed and follow the logic through with a debugger.

Focusrite

Exercise 3 / Hint 3

Check the `phaseInverted` bitset

`AudioProcessor::prepareState` takes the phase button states from the UI, and processes them into values the audio processor can use.

The processed phase values are stored in a bitset: are they being written correctly?

Focusrite

Exercise 3 / Answer

Swapped arguments invert Input 1

120 Hz start state	Expected	Broken
Left and right peaks	**0.4**	0.4
Output correlation	**+1**	**−1**
Mono peak	**0.4**	**0**

```
prepared.phaseInverted.set(input.phaseInverted, inputIndex);
```

`set` takes the position first. With both buttons off, Input 2 writes `true` to bit 0, so Input 1 is always inverted and PHASE 1 does nothing.

Exercise 3 / Fix

Store each flag at correct position

```
prepared.phaseInverted.set(inputIndex, input.phaseInverted);
```

- Check each PHASE button alone and both together.
- A per-channel test can't prove a mix is correct.
- Each channel peaked at 0.4; the mono sum was silent.

Optional catch-up: stop and close the app; build, test and open the solution.

```
cmake -DEX=3 -DMODE=solution -P scripts/exercise.cmake
```

After the break: close this app, then open Exercise 4.

```
cmake -DEX=4 -DMODE=start -P scripts/exercise.cmake
```

Break

15 minutes.

Exercise 4 / Bug report

A steady tone has a faint buzz

Reported setup: headphones, default signal and levels.

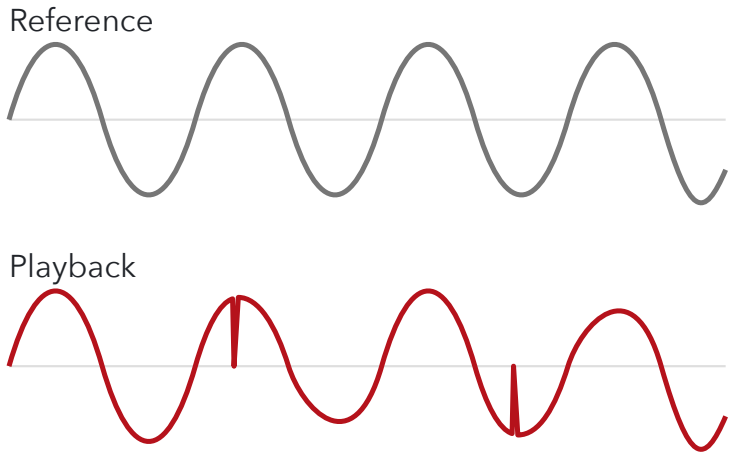
1. Start the supplied steady tone.
2. Try another buffer size in the device settings.
3. Switch back and compare the sound.

Expected: a clean, steady tone in either setting.

Actual: a faint buzz; its character changes with the setting.

Frequency: repeatable; CPU usage looks normal.

Steady tone and playback



Illustrative waveforms

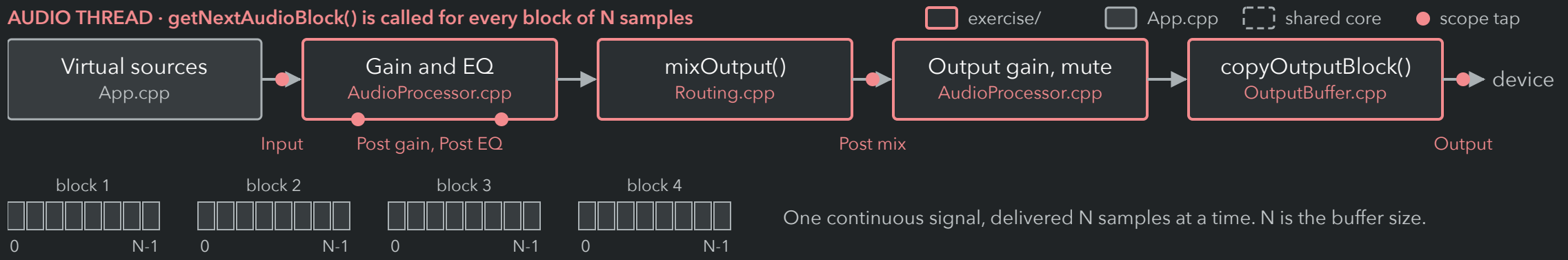
Block-based processing

- The audio subsystem asks for **N samples per callback**: the buffer size.
- Each callback must fill exactly N samples on every channel, at indexes 0 … N−1.
- A fault at every block end adds **spectrum lines spaced by the block rate**.
- Meeting every deadline doesn't mean the samples are right.

Buffer at 48 kHz	Block length	Blocks per second
128 samples	2.67 ms	375
256 samples	5.33 ms	187.5

Exercise 4 / Investigate

Find where delivered samples differ from intended



REPRODUCE

1. Default Sine, 0dB gain, flat EQ, mono off.
2. Listen at **128**, then **256** samples (DEVICE tab).
3. The audio output has visible notches in the waveform

THEORY

- Each callback delivers exactly N samples.
- Block-end faults add lines spaced f_s / N.

After each edit: `cmake -P scripts/check.cmake` or rebuild the app.

Exercise 4 / Hint 1

Spikes in the scope change based on buffer size

We've established that the buzzing sound seems to change when the buffer size changes.

Try using the scope to see the buzzing visually. How does the waveform change when the buffer size changes?

Exercise 4 / Hint 2

Check the output buffer

The buzzing only seems to appear at the output stage.

Find where the output samples are processed, and see if anything looks amiss. What could be causing a cyclical issue like this?

Focusrite

Exercise 4 / Hint 3

How many samples are copied each time the buffer is processed?

Take a look at `copyOutputBlock` in `OutputBuffer`. Here, audio samples are being copied from the source to the destination buffer.

Each time the buffer is copied, we seem to be losing a sample. What could be causing that?

Focusrite

Exercise 4 / Answer

The copy omits one requested sample

Block size	Wrong zero-based positions	Repeats
128	**127, 255, 383, …**	375 / s
256	**255, 511, 767, …**	187.5 / s

```
std::copy_n(source.begin(), sampleCount - 1, destination.begin());
```

The destination is cleared, then only **N − 1** samples are copied, so the last sample of every block drops from 0.25 to zero. Callback timing was fine throughout.

Exercise 4 / Fix

Copy the complete requested range of samples

```
std::copy_n(source.begin(), sampleCount, destination.begin());
```

- Test empty, one-sample, odd and partial final blocks.
- A sample count is not a last index.
- No deadline misses, yet every 128th sample was zero.
- Patterns in waveforms can be a useful debug tool.
- Compare with samples you worked out yourself.

Optional catch-up: stop and close the app; build, test and open the solution.

```
cmake -DEX=4 -DMODE=solution -P scripts/exercise.cmake
```

Next: close this app, then open Exercise 5.

```
cmake -DEX=5 -DMODE=start -P scripts/exercise.cmake
```

Exercise 5 / Bug report

My saved setup sounds different

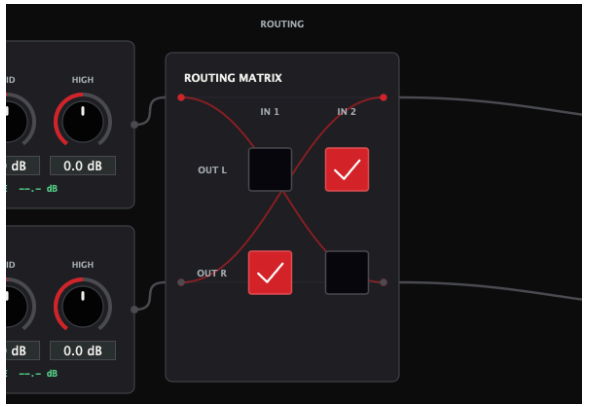
Reported setup: existing preset; original version unknown.
User provided: the preset file (`legacy-v1.json`) and a screenshot showing their settings when they saved.

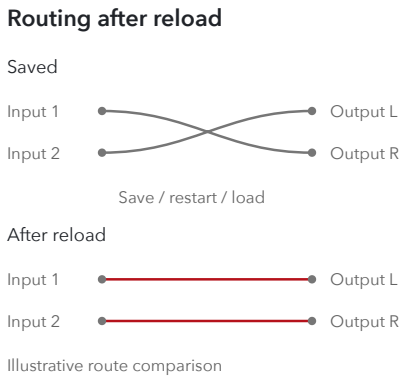
1. Open the app and click 'Load Preset'
2. Navigate to the user's preset file, in `assets/presets`
3. Play the audio and listen to each output

Expected: the saved listening setup is restored.

Actual: audio comes from different outputs.

Frequency: repeatable every time with this preset; others unchecked.





Exercise 5 / Theory

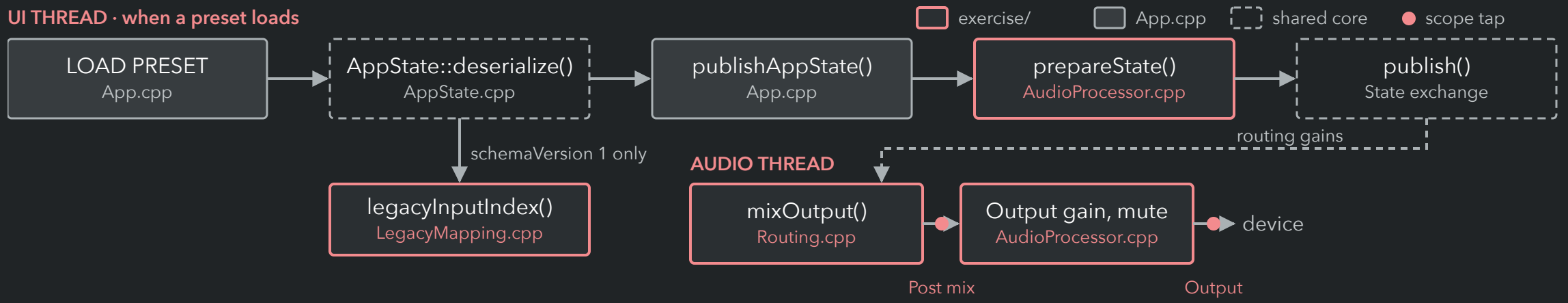
Versioned saved state

- Preset files are authored by a class in the app. Each **schema version** has its own definition.
- On load: read the version, **migrate** to the current model, **validate**, then publish.
- Valid JSON doesn't prove the values were read with the right meaning.
- A save and reload only shows the current format agrees with itself.

Version	How routing is stored
1	`outputChannelMappings: [2, 1]`, the input channel for Output L and Output R
2 (current)	A gain matrix: rows are outputs, columns are inputs

Exercise 5 / Investigate

Find where the loaded setup stops matching



REPRODUCE

1. Save and reload a current preset; check **DIFF**.
2. Restart; load `assets/presets/legacy-v1.json`.
3. Listen to each output: Input 1 is 1 kHz, Input 2 is 250 Hz.
4. Expected: Input 2 → Output L, Input 1 → Output R.

After each edit: `cmake -P scripts/check.cmake` or rebuild the app.

THEORY

- v1 `[2, 1]`: the input channel for Output L, R.
- Migrate old versions, validate, then publish.

Exercise 5 / Hint 1

Do a roundtrip with a newly saved preset

Set up the customer's routing setup in the app, then save a new preset with those settings.

When you load this new preset back in, does it behave the same way as the customer preset? Take a look at the preset files and inspect the differences.

Focusrite

Exercise 5 / Hint 2

Presets in the old schema need to be updated to work with this version of the app

The user's preset is using an old schema, and the app needs to update it to be compatible with the way the state is now represented.

Look for somewhere in the code where this updating might be taking place, and carefully inspect the logic.

Focusrite

Exercise 5 / Hint 3

The channel ids used to be one-based

Take a look at `LegacyMapping.cpp`. This is where legacy preset files are updated to work with the current version of the app.

In the old preset schema, the `outputChannelMappings` were one-based. Is this being respected in the update logic?

Focusrite

Exercise 5 / Answer

A channel number used as an index

Representation	Value	Meaning
Version 1 channel number	2	Input 2
Current array index	1	Input 2
Broken wrapped index	0	**Wrongly selects Input 1**

```
return static_cast<std::size_t>(oneBasedInput) % inputCount;
```

The range check (1 or 2) passes, then `% 2` turns channel 2 into index 0.

Exercise 5 / Fix

Convert, then index: version 1 only

```
return static_cast<std::size_t>(oneBasedInput - 1);   // after the 1–2 range check
```

- Handle the translation as a separate step
- Reject invalid state before loading or define fallback behaviour.
- Test current, legacy versions and bad versions.

Optional catch-up: stop and close the app; build, test and open the solution.

```
cmake -DEX=5 -DMODE=solution -P scripts/exercise.cmake
```

Next: close this app, then open the combined exercise.

```
cmake -DEX=final -DMODE=start -P scripts/exercise.cmake
```

Combined exercise / Bug report

My recording sounds wrong

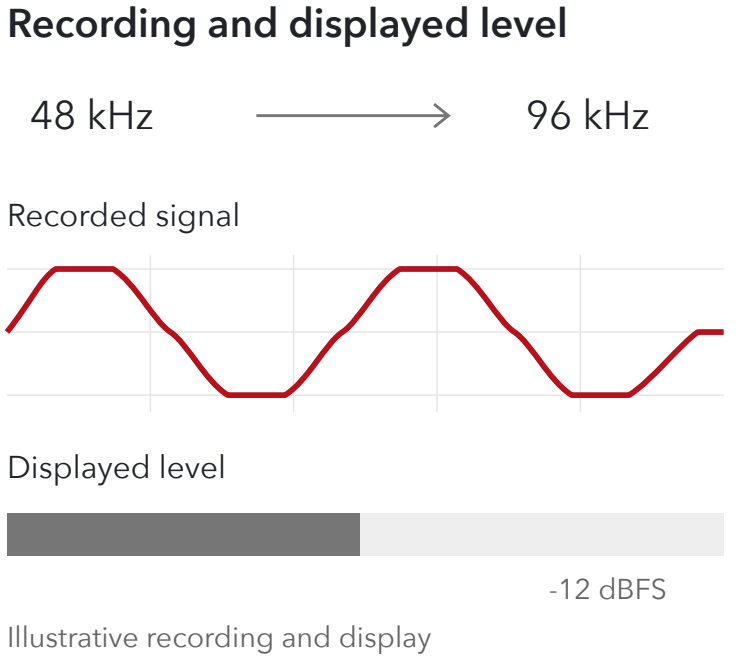
Reported setup: internal loopback recording.

1. Start playback and a loopback recording at 48 kHz.
2. Change the app to 96 kHz during the session.
3. Stop recording and listen to the captured audio.

Expected: the recording matches clean playback.

Actual: distorted recording, and playback crackles while recording; the capture meter looks normal.

Frequency: reported after rate changes; trigger not isolated.



Combined exercise / Theory

Sample rates, meters and callback budgets

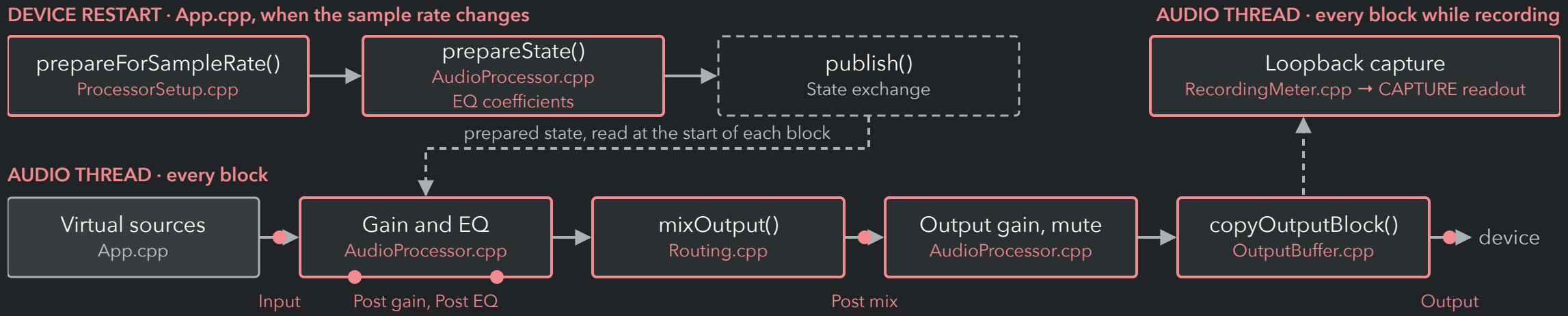
- Filter coefficients are designed for one sample rate: $\omega_0 = 2\pi f_0 / f_s$, so at another rate the filter moves in Hz.
- Anything prepared for a rate must be rebuilt when it changes.
- A meter only represents the point in the signal chain it reads.
- Each callback must finish within N / f_s: **1.33 ms** at 96 kHz with 128 samples.

Designed at	Runs at	Centre frequency
48 kHz	48 kHz	f_0
48 kHz	96 kHz	$2f_0$

Focusrite

Combined exercise / Investigate

Find the distortion, the misleading meter and the glitches



REPRODUCE

1. Play at **48 kHz**; enable **LOOPBACK RECORD**.
2. DEVICE tab: **96 kHz**; watch **MAX** and **MISSES**.
3. Stop recording; **REPLAY CAPTURE** at the same rate.
4. No 96 kHz? The focused check reproduces it.

THEORY

- Coefficients only suit their design rate.
- A meter only describes the tap it reads.
- Budget at 96 kHz, 128 samples: 1.33 ms.

After each edit: `cmake -P scripts/check.cmake` or rebuild the app.

Focusrite

Combined exercise / Answer

Stale coefficients, a pre-DSP meter and a slow count

96 kHz, 128-sample blocks	Baseline	Broken
Recording-bus peak	0.736	1.194
Displayed capture peak	0.736	0.300
Clip count per callback, full take	< 0.01 ms	2.4 ms

1: EQ coefficients stay prepared for 48 kHz, so the band moves up and 240 Hz clips.

2: the capture meter reads the input, before the DSP, so it shows the unchanged source.

3: the clip count recounts the whole take every block, overrunning the 1.33 ms budget.

Combined exercise / Fix

Three fixes in two files

1 · `exercise/ProcessorSetup.cpp`, `prepareForSampleRate()`: prepare for the device's rate.

```
return AudioProcessor::prepareState(state, sampleRate);
```

2 · `exercise/RecordingMeter.cpp`, `recordingMeterPeak()`: meter the recorded bus.

```
return analyze(recordingBus).peak;
```

3 · `exercise/RecordingMeter.cpp`, `updateCaptureClipCount()`: count only the new frames.

```
const auto first = capture.frameCount() - std::min(newFrames, capture.frameCount());
for (std::size_t output = 0; output < outputCount; ++output)
    for (const auto sample : capture.channel(output).subspan(first))
        if (std::abs(sample) >= 1.0f)
            ++clipsSoFar;
return clipsSoFar;
```

Combined exercise / Fix

Rebuild the EQ, meter the correct bus, count only new frames

- Rebuild coefficients on every sample rate change.
- Count only the new block's clips; keep a running total.
- Meter the bus that reaches the recording.
- Test 44.1, 48 and 96 kHz, repeated changes and long recordings.

Optional catch-up: stop and close the app; build, test and open the solution.

```
cmake -DEX=final -DMODE=solution -P scripts/exercise.cmake
```

Focusrite

Summary

Six bug reports, six issues solved

Customer report	Fixed in	The root cause
The test tone sounds distorted	`Gain.cpp`	Amplitude decibels use 20 $\log_{10}$, not 10
No sound in the right channel	`Routing.cpp`	The matrix is stored [output][input]
No sound when in mono	`AudioProcessor.cpp`	`bitset::set` takes the position first
A steady tone has a faint buzz	`OutputBuffer.cpp`	A sample count is not a last index
My saved setup sounds different	`LegacyMapping.cpp`	Version 1 channel numbers start at 1
My recording sounds wrong	`ProcessorSetup.cpp` `RecordingMeter.cpp`	Prepared state follows the rate; a meter shows the correct tap; every block takes the same time

Focusrite

Summary

Following a structure to finding issues can really help break down a problem.

01

Find the first bad stage

Reproduce, then measure at each stage against values you worked out yourself.

02

Make it explicit

Units, index order, numbering, sample rate and per-block cost. Improve your code using well-defined names, types and tests.

03

Repeatable tests to verify functionality

A regression test that fails as you add functionality that covers the edge cases which are time consuming to test.

Resources

Keep going

The app, every exercise, hints and solutions:
github.com/FocusriteGroup/the-audio-interface-escape-room-adc-26

- Finish the exercises you didn't reach. Each starts on its own: `-DEX=1` to `-DEX=5`, or `-DEX=final`.
- Start one again from its original code with `-DRESET=ON`.
- Open the solution with `-DMODE=solution`.

Thank you

Any questions?

Ross Chisholm and Alex Middleton-Dalby

The app, exercises and solutions:
github.com/FocusriteGroup/the-audio-interface-escape-room-adc-26